

Lista de Exercícios – MiniCCO-1

Escala dos Raios e Otimização Geométrica de Bifurcações

Disciplina: Algoritmos e Programação Avançada
Prof. Dr. Rafael Alves Bonfim de Queiroz

2026.1

1 Objetivo

O objetivo desta atividade é dar continuidade ao exercício MiniCCO-0, incorporando dois elementos fundamentais do método CCO:

1. o ajuste ou escala dos raios dos segmentos da árvore arterial;
2. a otimização geométrica da posição de uma nova bifurcação.

No MiniCCO-0, a árvore arterial simplificada foi construída considerando pontos, segmentos, testes geométricos, função custo geométrica simples e seleção de conexões candidatas. Nesta nova etapa, o aluno deverá enriquecer o modelo, associando propriedades físicas aos segmentos, em especial raio, comprimento, resistência hidráulica, fluxo e volume intravascular.

A função custo principal a ser minimizada será o volume intravascular total da árvore.

2 Descrição Geral do Problema

Considere uma árvore arterial simplificada em 2D, construída dentro de um domínio circular, como no MiniCCO-0.

Ao inserir um novo terminal P_{novo} , o programa deve:

1. escolher uma conexão candidata com um segmento existente;
2. criar temporariamente uma nova bifurcação;
3. ajustar os raios dos segmentos segundo uma lei de bifurcação;
4. calcular o volume intravascular total da árvore;
5. otimizar geometricamente a posição da bifurcação dentro da região triangular formada pelos três pontos envolvidos;
6. aceitar a configuração que produzir o menor volume intravascular válido.

3 Contexto Computacional

A árvore deve continuar sendo representada por uma estrutura de árvore binária, mas agora cada nó ou segmento deve armazenar informações adicionais.

Sugere-se estender a estrutura usada no MiniCCO-0:

```
typedef struct Point {
    double x;
    double y;
} Point;

typedef struct No {
    struct No *esq;
    struct No *dir;
    struct No *pai;

    int id;

    Point p;           // coordenada distal do
                       segmento
    double raio;       // raio do segmento
    double comprimento; // comprimento do segmento
    double fluxo;      // fluxo no segmento
    double resistencia; // resistencia hidraulica
    double volume;     // volume intravascular do
                       segmento

    int qtd_term_distal; // quantidade de terminais
                       distais
} No;

typedef No* ptrNo;
```

4 Parâmetros Físicos Sugeridos

Utilize os seguintes parâmetros:

- Q_{perf} : fluxo total de perfusão;
- p_{perf} : pressão de perfusão;
- p_{term} : pressão terminal;
- μ : viscosidade sanguínea;
- γ : expoente da lei de bifurcação;
- N_{term} : número de segmentos terminais.

Valores sugeridos:

$$\begin{aligned}
Q_{perf} &= 8.33 \times 10^{-6} \text{ m}^3/\text{s}, \\
p_{perf} &= 1.33 \times 10^4 \text{ Pa}, \\
p_{term} &= 7.98 \times 10^3 \text{ Pa}, \\
\mu &= 3.6 \times 10^{-3} \text{ Pa} \cdot \text{s}, \\
\gamma &= 3.
\end{aligned}$$

Para o caso de fluxos terminais iguais:

$$Q_{term} = \frac{Q_{perf}}{N_{term}}.$$

5 Parte A – Comprimento, Resistência e Volume

Para cada segmento j , calcule seu comprimento:

$$l_j = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}.$$

A resistência hidráulica deve ser calculada pela Lei de Poiseuille:

$$R_j = \frac{8\mu l_j}{\pi r_j^4}.$$

O volume intravascular de um segmento cilíndrico é:

$$V_j = \pi r_j^2 l_j.$$

O volume total da árvore é:

$$V_{total} = \sum_{j=1}^{N_{seg}} \pi r_j^2 l_j.$$

Tarefa

Implemente as seguintes funções:

```

double distancia(Point a, Point b);

double calculaComprimento(ptrNo seg);

double calculaResistencia(double mu, double comprimento,
                           double raio);

double calculaVolume(double comprimento, double raio);

double calculaVolumeTotal(ptrNo raiz);

```

6 Parte B – Contagem de Terminais Distais

Para ajustar fluxos e raios, é necessário saber quantos segmentos terminais existem distalmente a cada segmento.

Implemente uma função recursiva que percorra a árvore em pós-ordem:

```
int atualizaQtdTerminaisDistais(ptrNo no);
```

A função deve obedecer à seguinte lógica:

- se o nó é terminal, então:

$$qtd_term_distal = 1;$$

- caso contrário:

$$qtd_term_distal = qtd_term_distal(esq) + qtd_term_distal(dir).$$

7 Parte C – Fluxo em Cada Segmento

Assumindo fluxos terminais iguais, o fluxo em um segmento deve ser proporcional ao número de terminais distais:

$$Q_j = qtd_term_distal(j) \cdot Q_{term}.$$

Tarefa

Implemente:

```
void atualizaFluxos(ptrNo no, double Qterm);
```

8 Parte D – Lei de Bifurcação e Escala dos Raios

Em cada bifurcação, os raios devem obedecer à lei:

$$r_p^\gamma = r_1^\gamma + r_2^\gamma,$$

onde:

- r_p é o raio do segmento pai;
- r_1 e r_2 são os raios dos segmentos filhos;
- γ é o expoente de bifurcação.

Neste exercício, utilize uma versão simplificada baseada nos fluxos:

$$r_j = C \cdot Q_j^{1/\gamma},$$

onde C é uma constante de escala.

Para simplificação inicial, adote $C = 1$ e depois normalize todos os raios pela raiz.

Opcionalmente, o aluno pode definir o raio da raiz a partir da resistência global desejada:

$$\Delta p = p_{perf} - p_{term}.$$

Tarefa

Implemente:

```
void atualizaRaiosPorFluxo(ptrNo no, double gamma);

void atualizaGeometriaFisica(ptrNo raiz,
double Qterm,
double gamma,
double mu);
```

A função `atualizaGeometriaFisica` deve:

1. atualizar quantidade de terminais distais;
2. atualizar fluxos;
3. atualizar raios;
4. atualizar comprimentos;
5. atualizar resistências;
6. atualizar volumes.

9 Parte E – Função Custo: Volume Intravascular

A função custo deste exercício será o volume intravascular total:

$$J = V_{total} = \sum_{j=1}^{N_{seg}} \pi r_j^2 l_j.$$

O objetivo é encontrar a posição da bifurcação que minimiza J .

Tarefa

Implemente:

```
double funcaoCustoVolume(ptrNo raiz);
```

10 Parte F – Otimização Geométrica da Bifurcação

Considere uma conexão temporária envolvendo três pontos:

- A : ponto proximal do segmento antigo;
- B : ponto distal do segmento antigo;
- C : novo ponto terminal;
- X : nova posição candidata da bifurcação.

A bifurcação X deve ser procurada dentro da região triangular formada por A , B e C .

$$X \in \triangle ABC.$$

A estratégia sugerida é usar uma busca exaustiva em grade dentro do triângulo.

10.1 Coordenadas Baricêntricas

Um ponto X dentro do triângulo pode ser escrito como:

$$X = \alpha A + \beta B + \lambda C,$$

com:

$$\alpha + \beta + \lambda = 1,$$

e

$$\alpha, \beta, \lambda \geq 0.$$

10.2 Busca em Grade

Utilize dois laços para discretizar α e β :

```
for (int i = 0; i <= M; i++) {
    for (int j = 0; j <= M - i; j++) {
        alpha = i / (double) M;
        beta = j / (double) M;
        lambda = 1.0 - alpha - beta;

        X = alpha*A + beta*B + lambda*C;

        avaliar X;
    }
}
```

Quanto maior M , mais fina será a busca.

Valores sugeridos:

$$M = 10, 20, 50.$$

Tarefa

Implemente:

```
Point pontoBaricentrico(Point A, Point B, Point C,
double alpha, double beta, double lambda);

int pontoDentroTriangulo(Point A, Point B, Point C,
Point P);

Point otimizaBifurcacaoPorGrade(ptrNo raiz,
Point A,
Point B,
```

```
Point C,  
int M,  
double *melhorCusto);
```

11 Parte G – Avaliação de uma Bifurcação Temporária

Para cada posição candidata X dentro do triângulo:

1. altere temporariamente a posição da bifurcação;
2. atualize comprimentos;
3. atualize fluxos;
4. atualize raios;
5. calcule o volume total;
6. verifique se há interseção geométrica;
7. guarde o melhor valor de J .

A melhor posição será:

$$X^* = \arg \min_{X \in \triangle ABC} V_{total}(X).$$

12 Parte H – Comparação com o MiniCCO-0

Execute o programa usando a mesma configuração do MiniCCO-0 e compare:

- árvore sem escala dos raios;
- árvore com escala dos raios;
- árvore com escala dos raios e otimização geométrica.

Compare as seguintes métricas:

- comprimento total;
- volume intravascular total;
- raio médio;
- raio da raiz;
- número de interseções rejeitadas;
- tempo de execução.

13 Entrada do Programa

O programa deve receber, via linha de comando:

```
./minicco1 Nterm R gamma M
```

onde:

- **Nterm**: número de terminais;
- **R**: raio do domínio circular;
- **gamma**: expoente da lei de bifurcação;
- **M**: resolução da busca em grade no triângulo.

Exemplo:

```
./minicco1 50 0.05 3.0 20
```

14 Saídas Esperadas

O programa deve imprimir:

- número total de nós;
- número total de segmentos;
- número de terminais;
- comprimento total da árvore;
- volume intravascular total;
- raio da raiz;
- raio médio dos segmentos;
- número de conexões testadas;
- número de conexões rejeitadas;
- tempo de execução.

Além disso, deve exportar ao menos um arquivo contendo os segmentos da árvore com os campos:

```
id, pai, x0, y0, x1, y1, raio, comprimento, fluxo,  
resistencia, volume
```

O formato pode ser:

- **.csv**;
- **.txt**;
- **.vtk**.

15 Visualização

O aluno deverá atualizar o script de visualização em Python/PyVista usado no MiniCCO-0.

A visualização deve representar:

- os segmentos da árvore;
- os raios dos segmentos, se possível usando tubos;
- a raiz;
- os segmentos terminais.

Sugestão em Python:

```
import pyvista as pv
import pandas as pd
import numpy as np

dados = pd.read_csv("arvore.csv")

plotter = pv.Plotter()

for _, row in dados.iterrows():
    p0 = np.array([row["x0"], row["y0"], 0.0])
    p1 = np.array([row["x1"], row["y1"], 0.0])

    linha = pv.Line(p0, p1)
    tubo = linha.tube(radius=row["raio"])

    plotter.add_mesh(tubo)

plotter.show()
```

16 Testes Numéricos

Execute pelo menos os seguintes testes:

Teste	N_{term}	γ	M
1	10	3.0	10
2	30	3.0	20
3	50	3.0	20
4	50	2.7	20
5	50	3.0	50

17 Discussão Obrigatória no Relatório

O relatório deve conter uma discussão sobre:

1. Como a escala dos raios altera a árvore gerada?
2. O raio da raiz aumenta com o número de terminais? Por quê?
3. A otimização geométrica reduziu o volume intravascular?
4. Qual o impacto do parâmetro M na qualidade da solução?
5. Qual o impacto do parâmetro M no tempo de execução?
6. A função custo baseada em volume favorece árvores mais curtas, raios menores ou ambos?
7. Como a complexidade do algoritmo muda em relação ao MiniCCO-0?

18 Análise de Complexidade

Considere:

- N : número de segmentos da árvore;
- M : resolução da grade triangular;
- K : número de candidatos de conexão avaliados.

A busca por pontos dentro do triângulo possui aproximadamente:

$$O(M^2)$$

pontos candidatos.

Se, para cada ponto candidato, for necessário recalcular propriedades da árvore inteira, o custo pode ser:

$$O(M^2N).$$

Se forem avaliadas K conexões candidatas:

$$O(KM^2N).$$

O aluno deverá discutir se sua implementação recalcula toda a árvore ou apenas os segmentos afetados.

19 Entregáveis

Cada grupo deverá entregar:

- código-fonte completo em C ou C++;
- instruções de compilação;
- arquivos de saída gerados;
- script Python/PyVista para visualização;
- relatório técnico até 10 páginas.

20 Critérios de Avaliação

Item	Peso
Correção da escala dos raios	20%
Cálculo de comprimento, resistência e volume	15%
Otimização geométrica da bifurcação	25%
Integração com o MiniCCO-0	15%
Visualização e arquivos de saída	10%
Relatório e análise crítica	15%

21 Observações Importantes

- Esta atividade não exige a implementação completa do CCO clássico.
- A otimização geométrica pode ser feita por busca exaustiva em grade.
- Não é necessário utilizar bibliotecas externas de otimização.
- O objetivo é compreender a relação entre estrutura de dados, geometria, escala dos raios e função custo.
- O código desenvolvido nesta atividade poderá ser reutilizado no trabalho prático final da disciplina.

22 Extensões Opcionais

Como atividades extras, o grupo pode implementar:

- comparação entre diferentes valores de γ ;
- função custo alternativa combinando volume e penalização angular;
- busca local refinada em torno do melhor ponto encontrado na grade;
- exportação em formato VTK;
- visualização dos raios como tubos no PyVista;
- comparação entre busca em grade e busca gulosa simples;
- paralelização da avaliação dos pontos candidatos.

23 Síntese

Este exercício representa uma etapa intermediária entre o MiniCCO-0 e o projeto final da disciplina. Ao final desta atividade, o aluno deverá possuir uma implementação capaz de:

- construir uma árvore arterial simplificada;
- associar raios, fluxos, resistências e volumes aos segmentos;
- aplicar uma lei de escala dos raios;
- calcular volume intravascular;
- otimizar geometricamente a posição de uma bifurcação;
- visualizar a árvore com informação geométrica e física.